

PROJECT 1

BOB JENKINS'S one_at_a_time HASH FUNCTION

SUBMITTED BY:

*Ala-an, Princess Beverly R.
Aliñgasa, Earl James Williams A.*

SUBMITTED TO:

Mrs. Regien Bacabis-Nakazato

September 15, 2024

TABLE OF CONTENTS

TITLE PAGE..... 1

TABLE OF CONTENTS..... 2

1 INTRODUCTION..... 3

 1.1 Introduction to the Algorithm..... 3

 1.2 Problem Statement..... 3

 1.3 Objective..... 3

2 LITERATURE REVIEW..... 3

 2.1 Background and Motivation..... 3

 2.2 Supporting Literature..... 4

3 METHODS..... 4

 3.1 Implementation Details..... 4

 3.2 Time Complexity..... 6

 3.2.1 Base Runtime..... 6

 3.2.2 Asymptotic Analysis..... 7

4 COMPARATIVE ANALYSIS..... 7

 4.1 Comparison with Other Algorithms..... 7

 4.1.1 Introducing Additive Hash..... 7

 4.1.2 Speed Comparison..... 8

 4.1.3 Memory Usage Comparison..... 8

 4.1.4 Local Collision Rate Testing..... 8

 4.1.5 Jenkins’s Own Tests..... 9

 4.2 Results and Findings..... 9

5 CONCLUSION..... 10

 5.1 Summary of Findings..... 10

 5.2 Real-World Applications..... 10

6 REFERENCES..... 10

1 INTRODUCTION

1.1 Introduction to the Algorithm

The *one_at_a_time hash function* (simply referred to as *one_at_a_time*) is a simple hash function that was created in 1997 by Bob Jenkins. Like many other hash functions, *one_at_a_time* takes a string-type key and outputs a non-negative index. It is non-cryptographic, lightweight, easy to implement, and has good distribution in spite of its simplicity - making it ideal for small-to-medium scale applications.

1.2 Problem Statement

For simple, non-cryptographic hash functions, security and scalability may not be a priority, but good, uniform, non-normal distribution of key-generated indices is what defines them. A good distribution of indices ultimately determines the operating speed and memory usage of such functions. This is because good distribution reduces collisions, thereby reducing collision handling, thereby reducing memory usage and instruction cycles for those cases.

1.3 Objective

one_at_a_time is designed to be simple to implement, limited in instruction count, and capable of mixing values well enough to induce some form of *avalanche behavior*. Avalanche behavior is exhibited when small changes to the key induce massive changes to the output index. *one_at_a_time* does this by iterative combining-mixing steps that scatter the bit values during hash-value computation. Through this, the function is able to be simple but effective in distributing key-generated indices for a hash-map/hash-table.

2 LITERATURE REVIEW

2.1 Background and Motivation

Hashing ensures authenticity of data. In modern cryptography, it serves as a fundamental tool in information security to authenticate transactions, messages, and digital signatures (Loo, n.d.). However, some hashing techniques focus solely on data management rather than cryptography. *one_at_a_time* is one such function.

one_at_a_time operates in three stages. In the first stage, it initializes a 32-bit, all-zero hash-value. In the second stage, it iteratively mixes each character or byte of the input into the hash through bitwise operations, altering the hash's state to incorporate the input data. Finally, in the third stage, it performs a final mixing of the hash-value to ensure that all input bits

influence the final hash-value. This function is exclusive for strings and character data to produce a hash that uniquely represents the input.

What sets `one_at_a_time` apart from other simple hash functions is its robust bitwise mixing, making it versatile and reliable for hashing small to medium-sized data sets. The robustness of its bitwise mixing enables decent avalanche behavior, making it perform very well despite the simplicity of its design. So although it may not be complex and offer encryption, it fulfills its purpose of evenly distributing indices across a hash-map/hash-table.

There are other simple hash functions that are known for their simplicity, effectiveness, relative speed, or uniformity. Such functions include DJB2 Hash, SDBM Hash, FNV-1 and FNV-1a Hash, ROT13 Hash, and the Additive hash function. However, most functions demonstrate the possibility of high collisions, thus allocating more space for chaining or other collision handling methods. Relatively strong avalanche behavior from `one_at_a_time` allows it to be almost as fast as the others, but with better index distribution.

2.2 Supporting Literature

For a simple hash function that is non-cryptographic, `one_at_a_time` demonstrates efficiency in terms of distributing key-generated indices uniformly, reducing the likelihood of two different keys producing the same hash (Jenkins, 2013). `one_at_a_time`, however, is vulnerable in terms of security, but this may not be an issue as long as it is only utilized for non-critical operations. Security aside, the base function does not account for limited hash-map/hash-table sizes. This can easily be resolved by adding a modulo to the equation for the final hash-value, producing an overall good distribution within bounds (Why Is There No Overflow Check in the Jenkins One-At-a-Time Hash Function?, 2024).

`one_at_a_time` was initially a variant created as part of Jenkins's research into efficient and reliable hash functions that could handle general-purpose hashing needs, such as hash-tables, lookup functions, and data storage (Jenkins, 2013). Although `one_at_a_time` was implemented in languages like PERL, it eventually became obsolete and was succeeded by the more efficient Lookup2 hash function (Akoto-Adjepong et al., 2020).

3 METHODS

3.1 Implementation Details

`one_at_a_time` is easy to implement due to its simplicity. The algorithm is short, straightforward, and does not require language-specific components to be implemented.

The algorithm requires the following for implementation:

1. Operability with 32-bit integers,
 2. Iterative structures such as `while` or `for`, and
 3. Bitwise operators, namely XOR (^), left bitwise shift (<<), and right bitwise shift (>>).
- This renders it feasible in practically any mainstream programming language.

The following is the pseudocode for `one_at_a_time`:

```

SET hash = 0

FOR letter IN key:
    hash = hash + letter
    hash = hash + (hash << 10)
    hash = hash ^ (hash >> 6)

hash = hash + (hash << 3)
hash = hash ^ (hash >> 11)
hash = hash + (hash << 15)

RETURN hash

```

The following is an implementation of `one_at_a_time` in C for the sake of demonstration:

```

unsigned int one_at_a_time(char key[], int key_length) {
    unsigned int hash = 0;

    for (int i = 0; i < key_length; i++) {
        hash += (unsigned int)key[i];
        hash += (hash << 10);
        hash ^= (hash >> 6);
    }

    hash += (hash << 3);
    hash ^= (hash >> 1);
    hash += (hash << 15);

    return hash;
}

```

For adaptability within any given hash-map/hash-table, the modulo operator is utilized to keep the hash-value within bounds. In the case of implementation within C, it may look something like this:

```

unsigned int one_at_a_time(char key[], int key_length) {
    unsigned int hash = 0;

    for (int i = 0; i < key_length; i++) {
        hash += (unsigned int)key[i];
        hash += (hash << 10);
        hash ^= (hash >> 6);
    }

    hash += (hash << 3);
    hash ^= (hash >> 1);
    hash += (hash << 15);

    hash %= (unsigned int)HASHTABLE_SIZE;
    if (hash < 0) {
        hash += (unsigned int)HASHTABLE_SIZE;
    }

    return hash;
}

```

This adds to the runtime of the function, but not by much.

3.2 Time Complexity

3.2.1 Base Runtime

The following is a breakdown of the runtime for each instruction within the pseudocode of `one_at_a_time`:

SET <i>hash</i> = 0	1
FOR <i>letter</i> IN <i>key</i> :	$n + 1$
<i>hash</i> = <i>hash</i> + <i>letter</i>	n
<i>hash</i> = <i>hash</i> + (<i>hash</i> << 10)	n
<i>hash</i> = <i>hash</i> ^ (<i>hash</i> >> 6)	n
 <i>hash</i> = <i>hash</i> + (<i>hash</i> << 3)	 1
<i>hash</i> = <i>hash</i> ^ (<i>hash</i> >> 11)	1
<i>hash</i> = <i>hash</i> + (<i>hash</i> << 15)	1
 RETURN <i>hash</i>	 1

The directly-computed time complexity of the pseudocode is given by $t(n) = 4n + 6$, although this may vary in other programming languages depending on the specific implementation

requirements. This runtime is the same for the best, worst, and average cases because every character of the key is consistently utilized for hash-value computation. There are no conditional statements (outside of the iterative structure) that introduce alternate behavior, and so for all cases, $t(n) = 4n + 6$.

3.2.2 Asymptotic Analysis

The following demonstrates the determination of upper-bounding function classes for $t(n)$:

$$t(n) \leq c \cdot g(n)$$

$$4n + 6 \leq c \cdot n$$

$$4 + (6/n) \leq c;$$

Taking the limit as n approaches infinity:

$$4 \leq c;$$

We can choose $c = 5$. For this value of c , we find the threshold value k as follows:

$$4n + 6 \leq 5n$$

$$6 \leq n,$$

$$\therefore k = 6.$$

We have found suitable values for c and k , hence we can say that $t(n) = O(n)$.

With similar reasoning and working, we find the following results for the other bounding function classes:

$$t(n) = \Omega(n) \quad \text{and} \quad t(n) = \Theta(n).$$

one_at_a_time is simply a linear-runtime function in every case.

4 COMPARATIVE ANALYSIS

4.1 Comparison with Other Algorithms

4.1.1 Introducing Additive Hash

The additive hash function (simply referred to as Additive Hash) is your typical, introductory hash function that can be easily implemented in every programming language. It takes a string-type key and returns a non-negative integer index.

The pseudocode for additive hash is as follows:

```
SET hash = 0
```

```
FOR letter IN key:
```

```
    hash = hash + letter
```

```
RETURN (hash % HASHTABLE_SIZE)
```

The C-implementation of this function is almost identical to the pseudocode.

4.1.2 Speed Comparison

Additive hash has a base runtime of $t(n) = 2n + 3$, exactly half that of the base runtime for `one_at_a_time`. However, in implementation, this may vary, but it can be said that additive hash is roughly twice as fast in most cases. It is worth noting that this does not account for time spent in collision handling.

4.1.3 Memory Usage Comparison

Due to additive hash's lack of bitwise mixing, there is no induced avalanche behavior. Hence it is expected to see higher collision rates in additive hash than in `one_at_a_time`. Higher collision rates require more allocated memory for chained keys. This means that additive hash is more wasteful than `one_at_a_time` and requires more memory when generating indices.

4.1.4 Local Collision Rate Testing

Utilizing implementations in C, we were able to hold equivalent hashing trials for both `one_at_a_time` and additive hash. The same conditions and keys were fed into both hash functions with their own empty hash-tables to populate. For various hash-table sizes, the number of collisions were counted. Some of the results are displayed in Figure 1.

HASHTABLE SIZE: 1000 one_at_a_time: 367 36.70% Additive Hash: 852 85.20%	HASHTABLE SIZE: 1000 one_at_a_time: 376 37.60% Additive Hash: 851 85.10%
HASHTABLE SIZE: 1100 one_at_a_time: 419 38.09% Additive Hash: 954 86.73%	HASHTABLE SIZE: 1100 one_at_a_time: 487 37.00% Additive Hash: 950 86.36%
HASHTABLE SIZE: 1200 one_at_a_time: 434 36.17% Additive Hash: 1048 87.33%	HASHTABLE SIZE: 1200 one_at_a_time: 440 36.67% Additive Hash: 1051 87.58%
HASHTABLE SIZE: 1300 one_at_a_time: 479 36.85% Additive Hash: 1147 88.23%	HASHTABLE SIZE: 1300 one_at_a_time: 493 37.92% Additive Hash: 1139 87.62%
HASHTABLE SIZE: 1400 one_at_a_time: 509 36.36% Additive Hash: 1244 88.86%	HASHTABLE SIZE: 1400 one_at_a_time: 499 35.64% Additive Hash: 1243 88.75%
HASHTABLE SIZE: 1500 one_at_a_time: 553 36.87% Additive Hash: 1346 89.73%	HASHTABLE SIZE: 1500 one_at_a_time: 546 36.40% Additive Hash: 1332 88.80%
HASHTABLE SIZE: 1600 one_at_a_time: 594 37.13% Additive Hash: 1448 90.50%	HASHTABLE SIZE: 1600 one_at_a_time: 613 38.31% Additive Hash: 1443 90.19%
HASHTABLE SIZE: 1700 one_at_a_time: 630 37.06% Additive Hash: 1539 90.53%	HASHTABLE SIZE: 1700 one_at_a_time: 622 36.59% Additive Hash: 1541 90.65%
HASHTABLE SIZE: 1800 one_at_a_time: 664 36.89% Additive Hash: 1643 91.28%	HASHTABLE SIZE: 1800 one_at_a_time: 657 37.06% Additive Hash: 1646 91.44%
HASHTABLE SIZE: 1900 one_at_a_time: 687 36.16% Additive Hash: 1736 91.37%	HASHTABLE SIZE: 1900 one_at_a_time: 687 36.16% Additive Hash: 1737 91.42%
HASHTABLE SIZE: 2000 one_at_a_time: 740 37.00% Additive Hash: 1841 92.05%	HASHTABLE SIZE: 2000 one_at_a_time: 736 36.80% Additive Hash: 1832 91.60%

Figure 1: (Left) Trials with 16-character keys and (Right) trials with 32-character keys

4.1.5 Jenkins's Own Tests

NAME	SIZE-1000	SPEED	INLINE	FUNNEL-15	FUNNEL-100	COLLIDE-32	COLLIDE-1000
Additive	1009	$5n+3$	$n+2$	15 into 2	100 into 2	37006	+806.02
Rotating	1009	$6n+3$	$2n+2$	4 into 1	25 into 1	24	+1.24
One-at-a-Time	1024	$9n+9$	$5n+8$	none	none	0	-0.05
Bernstein	1024	$7n+3$	$3n+2$	3 into 2	3 into 2	4	+1.69
FNV	1024	?	?	?	?	?	?
Pearson	1024	$12n+5$	$4n+3$	none	none	0	+1.65
CRC	1024	$9n+3$	$5n+2$	2 into 1	11 into 10	0	+0.07
Generalized	1024	$9n+3$	$5n+2$	none	none	0	-1.83
Universal	1024	$52n+3$	$48n+2$	4 into 3	50 into 28	0	+0.20
Zobrist	1024	$10n+3$	$6n+2$	none	none	1	-0.03
Paul Hsieh's	1024	$5n+17$	N/A	3 into 2	3 into 2	1	+1.12
My Hash	1024	$6n+35$	N/A	none	none	0	+0.33
lookup3.c	1024	$5n+20$	N/A	none	none	0	-0.08
MD4	1024	$9.5n+230$	N/A	none	none	1	+0.73

Figure 2: A Comparative Analysis Table of several hash functions (Jenkins, 2013)

Figure 2 shows the table of results from hash-function testing compiled by Bob Jenkins (2013). Some hash functions are listed along with their collision rates in columns 'COLLIDE-32' and 'COLLIDE-1000'. 'COLLIDE-32' indicates the number of collisions encountered when hashing English dictionary entries into a 32-bit index. Meanwhile, 'COLLIDE-1000' shows the χ^2 result when hashing the same dictionary to a hash-table of size 1000. A higher value indicates poorer distribution, leading to more collisions, which can affect the performance of data structures like hash tables.

In both metrics of COLLIDE-32 and COLLIDE-1000, one_at_a_time performs significantly better than additive hash.

4.2 Results and Findings

Under ideal conditions, additive hash would outpace one_at_a_time by a factor of two, but this may not be enough of a benefit for actual usage. As is shown in Figure 1, additive hash performs significantly worse than one_at_a_time when it comes to avoiding collisions, and this becomes worse for larger hash-tables. Additive hash may have the speed, but its usefulness versus one_at_a_time drops drastically since it cannot lower its memory usage.

The collision rates seem to stay roughly the same for one_at_a_time (highlighting its statistically uniform distribution capabilities), but they grow quickly for additive hash. Assuming a constant memory usage per collision, it can be estimated that additive hash uses roughly three times more memory for collision handling than one_at_a_time in the worst possible cases. For small-to-medium use cases, this is not feasible.

With speed and collision rates considered, it is very clear that one_at_a_time is the superior hash function for equivalent use cases.

5 CONCLUSION

5.1 Summary of Findings

one_at_a_time is simple, with only minor changes needed for implementation. In terms of advantages, it is quick, has decent avalanche behavior, and performs better than other simple hash functions such as additive hash. In terms of disadvantages, it lacks security via encryption, and may not be as uniformly distributed as more complex hash functions. For simple use cases, one_at_a_time serves its purpose quickly and can be utilized with little collision-handling needed.

5.2 Real-World Applications

Although one_at_a_time became obsolete, it is a simple version of Jenkins's set of hash functions that can be used in educational contexts. It is an easy and accessible example that helps illustrate fundamental concepts in hashing algorithms. It can also be used as a model to create a new hash function (Hsieh, 2024). Specifically, it can be applied for prototyping and experimentation to get a quick idea of how hashing works before moving on to more robust algorithms.

6 REFERENCES

Loo, A. (n.d.). Hash Function. Corporate Finance Institute.

<https://corporatefinanceinstitute.com/resources/cryptocurrency/hash-function>

Jenkins, B. (2013). A Hash Function for Hash Table Lookup. Burtleburtle.net.

<http://www.burtleburtle.net/bob/hash/doobs.html>

Akoto-Adjepong, V., Asante, M., & Okyere-Gyamfi, S. (2020). An Enhanced Non-Cryptographic Hash Function. *International Journal of Computer Applications*, 176(15), 10–17.

<https://doi.org/10.5120/ijca2020920014>

Why is there no overflow check in the Jenkins one-at-a-time hash function? (2024). Stack Overflow.

<https://stackoverflow.com/questions/70732034/why-is-there-no-overflow-check-in-the-jenkins-one-at-a-time-hash-function>

Hsieh, Paul. (2024). Azillionmonkeys.com.

<http://www.azillionmonkeys.com/qed/hash.html>